



Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

CS7CS4 Machine Learning

Week 9

Shane O'Donnell - 21364002

Contents

Introduction.....	1
Method.....	1
(i).....	1
(a) Dataset Description	1
(b) Model Downsize.....	2
(c) Two more different Downsize techniques and rationale.....	3
(d) effect of inclusion of bias terms in self-attention layers	5
(e) Impact of inclusion of skip connections	7
(ii) Model evaluation on test sets	8
(a) child test set – performance loss/compare to baseline – random weights model	8
(b) Shakespeare test set – performance loss/ compare to a baseline – random weights ..	9

Introduction

For this investigation a decoder only transformer model was as investigated to assess its ability to model natural language. For this investigation 3 datasets were provided; the training dataset was a child speech file while the test sets comprised of a child speech test set and a test set with excerpts from a Shakespeare play. For this investigation, different model architectures as well as the importance of the presence of bias terms and skip (residual) connections and the trained model's performance on the two test sets.

Method

(i)

(a) Dataset Description

Training Dataset

To utilise the training dataset provided the “open” function was altered to point to the training text file “input_childSpeech_trainingSet.txt”. From here it was possible to assess the size of the training set in terms of the number of unique characters and the vocabulary size. The training set included 246982 characters and had a vocabulary size of 40. The sentences in this set a short and simple and contains repetitive phrases.

Test Dataset

To analyse the characteristics of the test set the unique characters and vocabulary size were also logged using the addition of print statements within the provided code file. This set has a unique character amount of 24114 and a vocabulary size of 40. The style of language for this dataset is the same as the training set in which the sentence structure is childlike in nature and follow a simple structure with repetitive language

Shakespeare Play Dataset

The Shakespeare dataset contains 1115394 characters and a vocabulary size of 65 which is significantly bigger than the train and test datasets. This dataset contains dialogue from a Shakespeare play and contains long texts with more vast vocabulary and punctuation. Compared to the previous two datasets, this dataset contains characters and punctuations which don't appear in the child speech training and testing dataset. These new characters are outlined in the table below.

Table 1: Characters that are unique to the Shakespeare dataset

Uppercase	E F J K O P Q T V X Z
Lowercase	q x z
Punctuation	! ' , - . : ; ?
Symbols	\$ & 3

(b) Model Downsize

To downsize the model to a manageable size to train locally the hyperparameters were changed to match the following specifications seen in table 1. The batch size refers to the amount of training sequences processed in a singular updated, the larger the batch size, the smoother the gradient while also using more GPU memory for training. Block size represents the maximum context length in characters, when predicting the next character, the model only takes the last 256 characters into account for its prediction. Max iterations refers to the number of parameter update steps during the training process, more iterations can lead to higher training times but also better fit. Although it can also lead to overfitting. Learning Rate is the step size of the optimiser and controls how big the weight updates are per step, too large can lead to divergence while too small can lead to the model learning very slowly. Evaluation iterations refers to how many batches are sampled when calculating the loss for training and validation. Embedding sets the width of the model and refers to the hidden representation for each character and position. The number of attention heads sets how many parallel self attention heads the model has at each layer. The number of heads is important as by having multiple heads, different heads can focus on different patterns within the text sequence. The parameter which sets the depth of the model is the number of transformer blocks and refers to how many transformer blocks are stacked, more layers can lead to deeper processing. Dropout refers to the regularisation rate set for the model, dropout takes a probability value which sets the chance that certain activations will be randomly zeroed during training to reduce overfitting.

Table 2: Model 1 hyperparameters

Batch size	64
Block size	256
Max Iterations	5000
Learning Rate	3e-4
Evaluation Iterations	200
Embedding	128
Number of attention heads	4
Number of transformer blocks (depth)	3
Dropout	0.2

With these parameters set it was possible to train the model, with these parameters the model had a total parameter count of 0.636968 million parameters which satisfies the constraint of the model being below 1 million. This reduction from 10.769704 million was possible by reducing the embedding from 384 to 128, the number of heads from 6 to 4 and number of transformer blocks from 6 to 3. This reduced the parameter count immensely, most notably due to the embedding being decreased by a third, this hyperparameter in particular controls the width of every representation in the network. The majority of weight in the model correspond the embedding and linear layers, these aspects of the model scale in the manner of $n_{embedded}^2$ for the attention and feed forward layers or $n_{embedded} * vocab_size$ for the embedding table and output layers. In contrast, reducing the number of attention heads or transformer blocks reduces parameters more linearly. The chosen hyperparameters keep the overall transformer structure intact while bring the model's total parameters to the desired amount.

(c) Two more different Downsize techniques and rationale

Model 1

For the hyperparameters outlined above, the model was trained on the “input_childSpeech_trainingSet.txt” dataset. From the training output the model displayed an initial loss in step 0 of 3.7184 for training and 3.7170 for validation, at step 500 the loss for train and validation drop to 0.3397 and 0.3686 respectively and only deviate slightly from these values for future steps until training finishes at 5000 iterations. This shows that the model quickly learns the basic character statistics and simple patterns in the dataset. Although after this the training reaches a plateau for the architecture of the model. Throughout training the loss for train and validation are very similar at each step iteration which displays there is no sign of overfitting; this displays the model generalises well to the held-out data instead of memorising the training set.

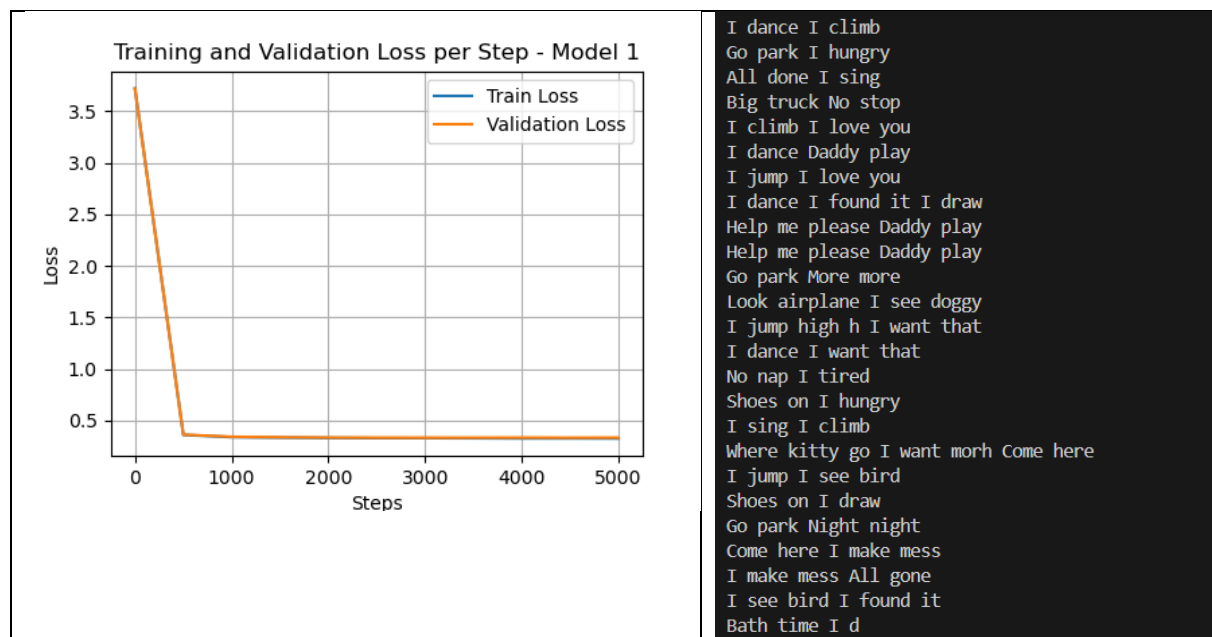


Figure 1: Model 1 training, validation loss and example output

The generated text from the trained model resembles the child like speech present in the training dataset, the sentences are short, simple, repetitive and in the present tense. The output lines resemble the same structure as the training set but offer unique sentences which suggests the model has learnt patterns rather than sentence strings. There are some misspelled tokens such as in the line “Where kitty go I want morh Come here” which displays the limitations of a small character amount in the training set. The repetition of “Daddy play” indicates the model has assigned a strong prior to the high frequency phrases.

Model 2 – Shallow and Wider

Model 2 uses the same dataset as model 1 and training set up but has wider embedding with the same number of heads but fewer transformer heads making the model shallower. This model has roughly the same total parameter count as model 1 (0.67172 million) to keep comparison as fair as possible. Model 2 offers a shallower but wider architecture. The chosen parameters to achieve this can be seen in table 3.

Table 3: Model 2 hyperparameters

Embedding	160
Number of attention heads	4

Number of transformer blocks (depth)	2
--------------------------------------	---

Compared to model 1, model 2 achieves a slightly lower loss in training but a slightly worse validation loss. At step 5000 the model achieved a training loss of [0.3237459659576416] and a validation loss of [0.33774203062057495]. The train and validation loss gap is still minimal meaning there is no clear sign of overfitting but the balanced configuration between depth and width for model 1 seems to generalise slightly better than model 2. From the outputted sentences from the model, the style is short, simple and child like which is similar to its training data and the model 1 output. Although, unlike model 1, this model was able to model sentences with 2 clauses such as “What is that I did it” and “Daddy read me dinosaur book before bed No nap” which shows this model had the ability to connect patterns together to form a singular sentence. Comparatively to model 1, this model also contains misspelling tokens such as “Daddy w bookie” which highlights the limitations of a smaller dataset. The repetition within the data set also causes frequent phrases to appear a lot due to the model assuming these tokens carry higher importance.

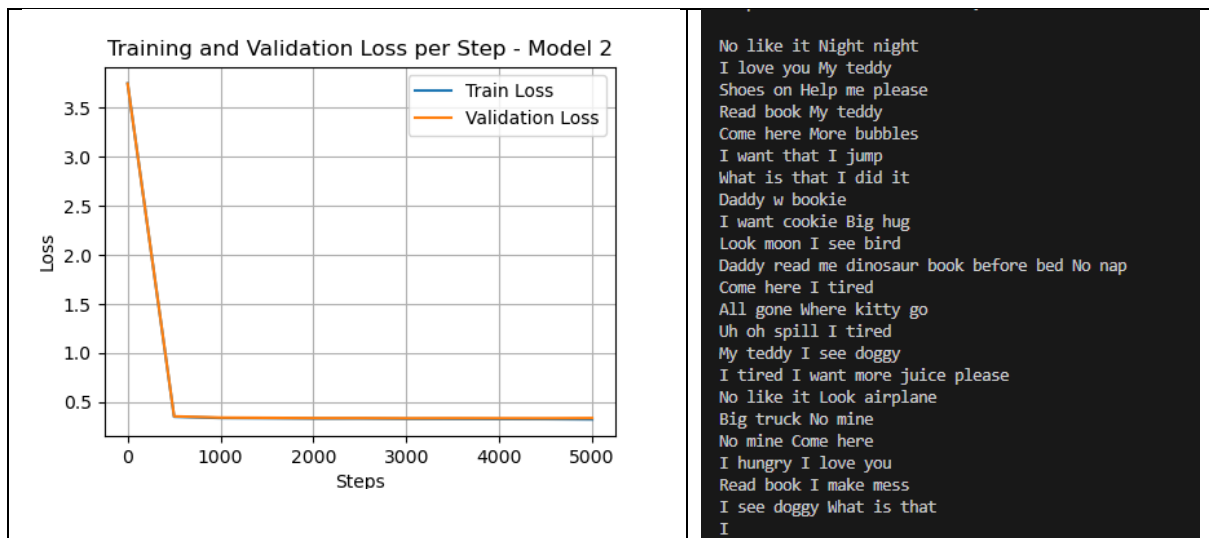


Figure 2: Model 2 train, validation loss and example output

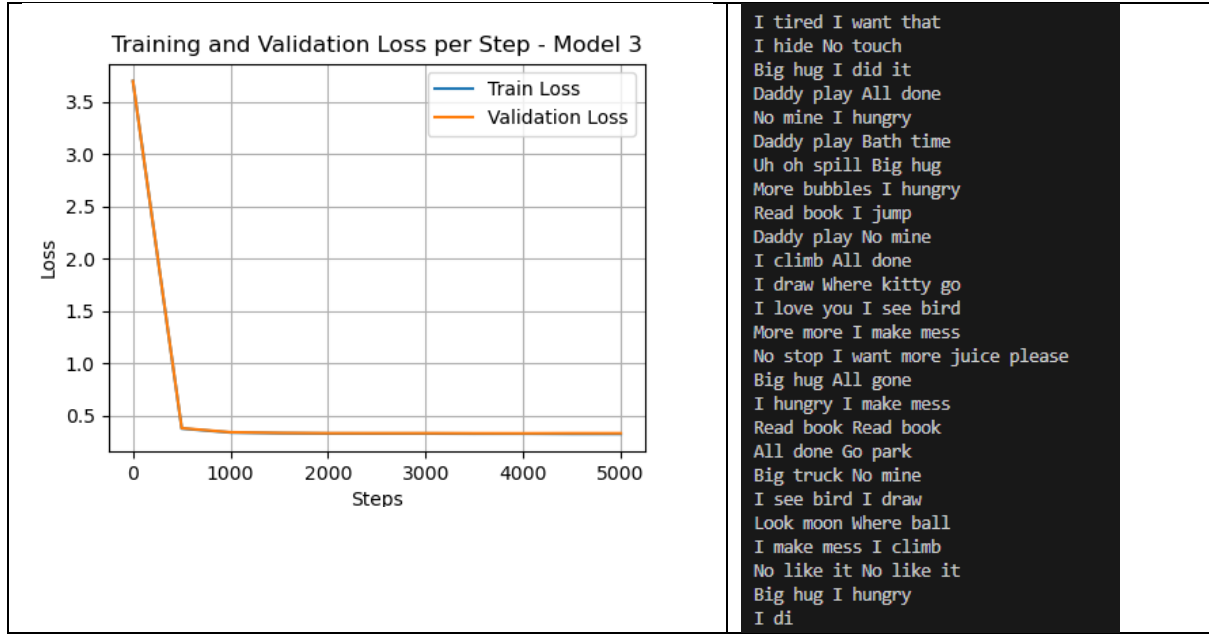
Model 3 – Deep and Narrow

The third model designed proposed a narrower embedding architecture but with more transformer heads to make the model deeper. This model had a total parameter count of 0.7035 million which closely approximates the last 2 models, only differing in architecture.

Table 4: Model 3 hyperparameters

Embedding	96
Number of attention heads	4
Number of transformer blocks (depth)	6

Model 3 achieved a final training loss of [0.32748138904571533] and validation loss of [0.33401620388031006] at step 5000. The training and validation loss closely approximate each other with validation being slightly higher [$\pm 6.53 \times 10^{-3}$], the small deviation does not justify the presence of overfitting.



It can be seen from the 3 models with varying architectures that the train and validate loss values achieved are extremely similar. It can be inferred that this is the case due to the simplicity of the dataset in size and the limited variability of the child speech corpus. In this investigation capacity is sufficient to fit the dataset so architectural reallocations (wider vs deeper) only bring marginal differences once training has converged. To balance the presence of embedding width and attention layers, I feel that for this investigation model 1 will be optimal; while having very similar results to the other models, it has a more balanced architecture in which embedding dimensions and the number of layers is matched. Each block has sufficient capacity to learn useful features to model child speech while not being too deep that optimisation becomes unstable or too wide that parameters are wasted on such a simple dataset.

(d) effect of inclusion of bias terms in self-attention layers

In self-attention layers, the addition of a bias term to the key, query, or value projections increases the model's flexibility overall by not solely relying on the linear combination of inputs but also on a learned offset. The computation in the linear layer can be computed using the following expression in which W is the weight matrix and b is the bias (learned offset per output dimension). Bias terms can shift the representation of each token independently of its exact input value. In theory this can increase flexibility as each neuron can shift its activation independently; combined with the non-linearity present in other parts of the network, this allows the model to represent a wider range of functions.

However the attention weights are obtained using a SoftMax over the dot product scores $q_i * k_j$. Due to the learned biases in the Q, K and V projections are not uniform constants, as in principle they can change relative scores between tokens. Although due to the small model and highly repetitive dataset adding these extra bias terms does not add a noticeable change to the learned attention patterns, which is consistent with the almost identical training and validation losses when bias is included or excluded.

$$y = Wx + b$$

For the previous training cycles, the attention projections are computed from the weight matrix alone. Although to demonstrate the difference, for this test the bias is enabled by altering the Head class in

the gpt.py file. From here the three models are trained again with the same parameter but with the bias enable on the self-attention layers.

From analysing the performance of the model with bias term inclusion, the training and validation loss is almost identical for both Model 1 variants. The only difference to be seen is the presence of longer strings in the sample output. Model 1 with no bias term displayed mostly short lines such as “I dance I climb” while the model 1 with bias inclusion was able to model longer strings which can be seen in figure 3.

The same behaviour is noted with the two other models with varied architecture. Model 3 takes slightly longer to full converge but all models reach approximately the same training and validation loss on convergence. The results suggest that for these small transformer models on a relatively simple character dataset containing a lot of repetition, the additional capacity provided by bias terms in the self-attention projections is fairly redundant. The models architecture and layers before this addition already has sufficient expressiveness.

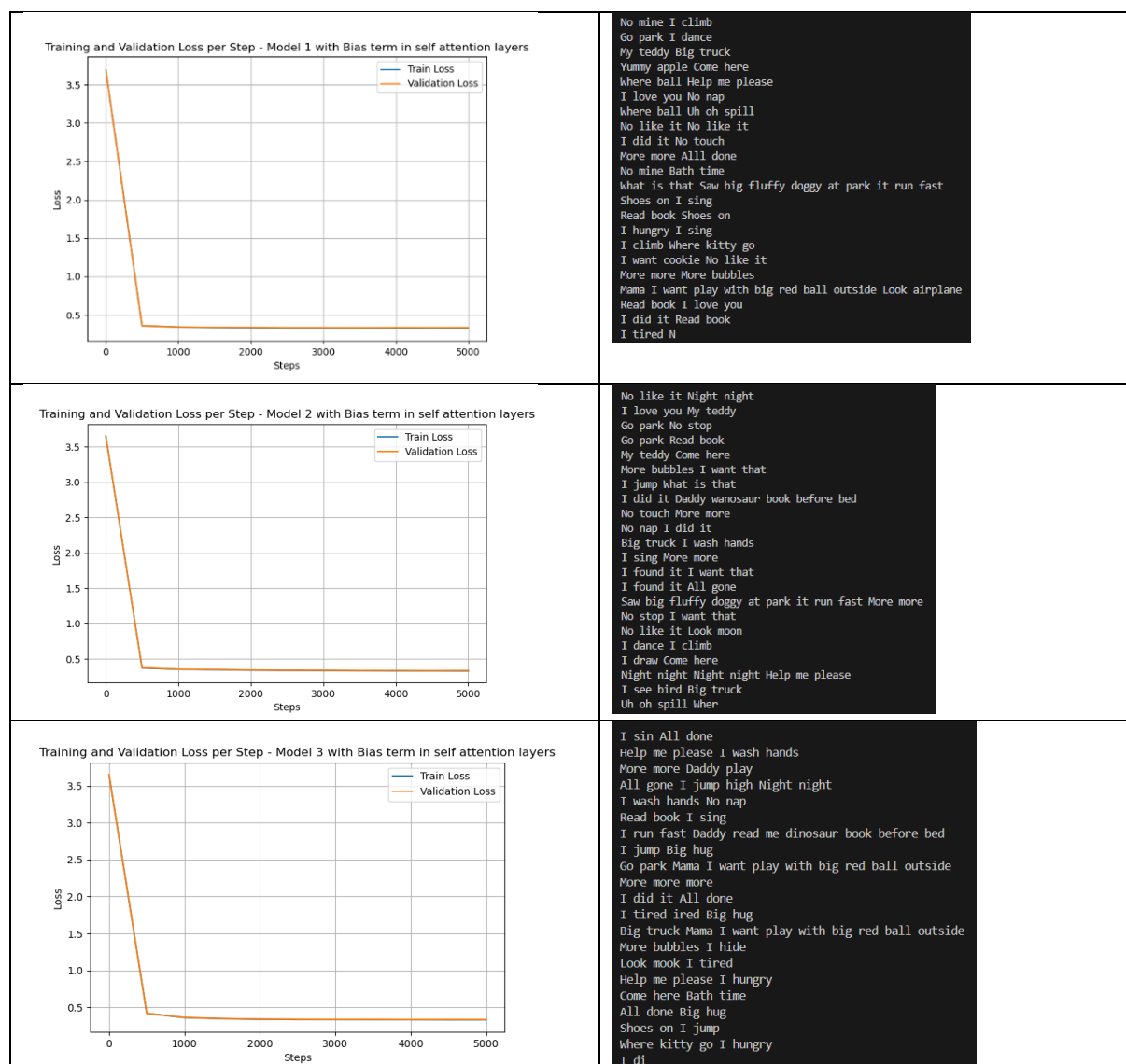


Figure 3: Training and Validation loss achieved by all 3 models with bias term inclusion

(e) Impact of inclusion of skip connections

Within transformers, skip connections (also known as residual connections) are fundamental in transformer design and allow for models to capture contextual relationships between tokens and words in a sequence. A transformers architecture includes many layers stacked on top of each other, including self-attention blocks which allow the model to assign weighted importance to different words and feed forward network blocks (FFN) which have the role of taking the contextual output from the self-attention blocks and transform them further to allow the model the ability to learn intricate patterns within the training dataset and improve its ability to model natural language.

In order to investigate the importance of skip connections, the model 1 trained and assessed in the earlier investigations was used as the baseline and two new models were trained with the same hyperparameters but disabling the skip connections in the self-attention blocks and the Feed Forward Network blocks independently to explore the models dependence on the presence of skip connections for each block type.

No skip connections in self-attention blocks

For this investigation skip connections were disabled for the self-attention blocks but kept for the feed forward network blocks. It can be seen that the loss per step for training and validation follow the same pattern which demonstrates that overfitting has not occurred. Although the training and validation loss eventually decreases after a long plateau between 1000 steps and 3500 steps, the models samples are unintelligible. The skip connections in the attention blocks allows the model to add corrections on top of a stable representation. Disabling these skip connections means the model must fully overwrite the representation each layer which weakens the gradient flow and hinders the models ability to model long range dependencies. At step 5000 the loss value for training and validation is logged at [2.044].

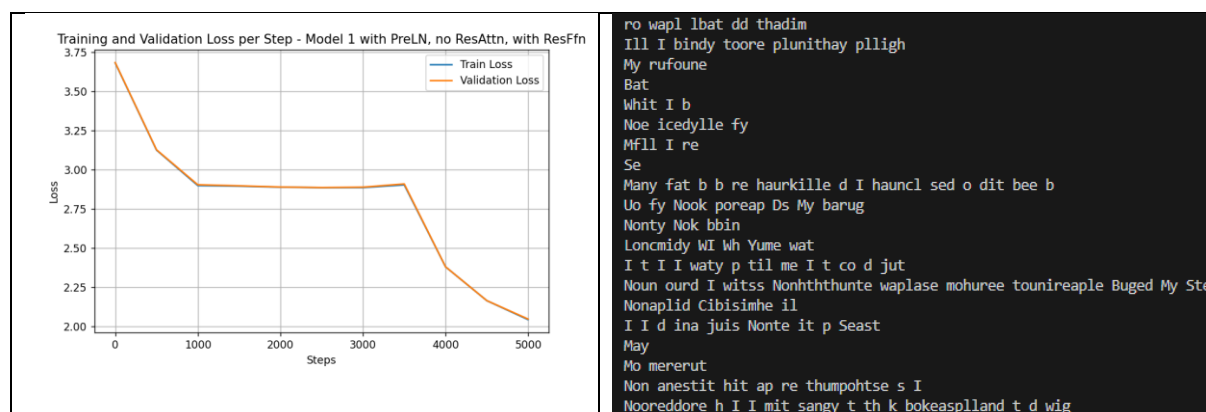


Figure 4: Model 1 with no residual connections in attention blocks

No skip connections in feed forward network blocks

For this investigation skip connections for the self attention blocks are enabled and the skip connections for the feed forward networks is disabled. From the loss graph, it can be seen that the loss achieved for training and validation are almost identical to each other and closely match the baseline metrics achieved when skip connections are enabled for both blocks. This demonstrates that the transformer model relies more on skip connections for the self attention blocks.

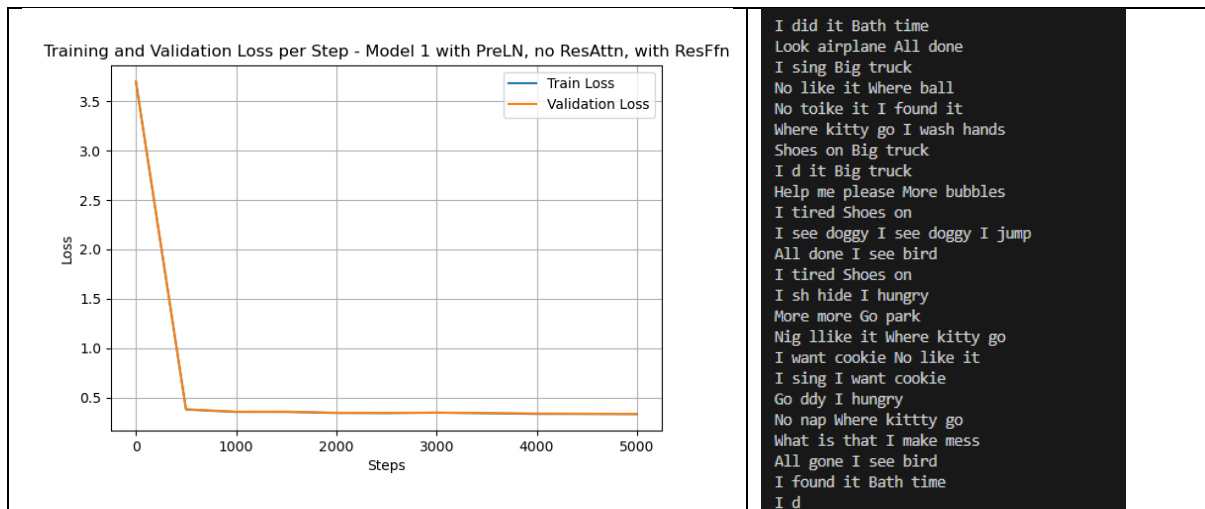


Figure 5: Model 1 with no residual connections in feed forward blocks

No Pre Normalisation

The final test in this section entailed disabling pre-normalisation (Pre-LayerNorm). The purpose of pre normalisation is to implement a design in which each transform block applies it to its input before the attention or feed forward sublayer to stabilise deep transformers during training by keeping the scale of inputs roughly constant which mitigates the issue of vanishing or exploding gradient. From the loss graph and example output from this model it can be seen that the model converges to approximately the same loss although it took an extra 500 steps. This displays that in small transformer model designs, pre normalisation is not as important as skip connections in the attention blocks, although in larger transformer models, small mismatches in scaling across layers can compound and vanishing or exploding gradients can be expected.

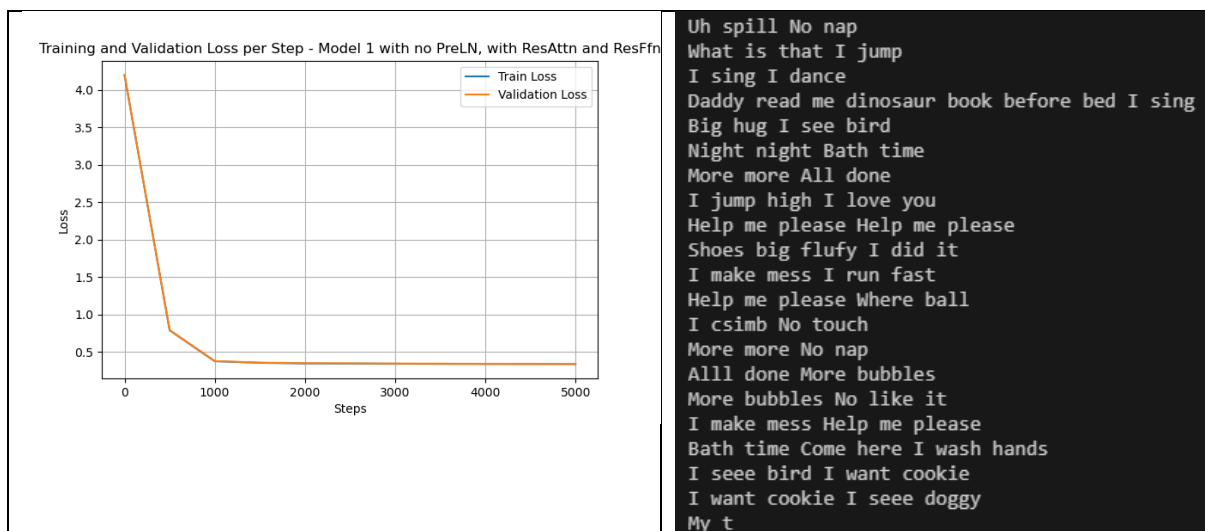


Figure 6: Model 1 with no pre normalisation

(ii) Model evaluation on test sets

(a) child test set – performance loss/compare to baseline – random weights model

The model 1 trained previously was tested on the child test set in order to assess it's ability to model child like speech, due to the test set having the same characters as well as a very similar

vocabulary the model performed with a similar loss achieved during training and validation of [0.339962970614433]. To compare the performance of the trained model, another model with the same hyperparameters was created but its weights were left at their initial random values due to not undergoing training. This model therefore has no knowledge of any dataset and will behave similarly to a uniform predictor over the vocab list. Table 5 displays the performance of both models on the same test set; it can be seen that the trained model outperforms the randomly weighted model which achieved a loss of [4.20233963727951] demonstrating that the randomised model was not able to model the child like speech present in the dataset. This loss value is as expected as it very closely approximates the log of the amount of character ($\log(65) = 4.17$) which confirms the hypothesis that the model is essentially guessing the next character with equal probabilities.

Table 5: Cross Entropy Loss for Model 1 tested on Child Speech Dataset

Trained Model Loss	0.339962970614433
Random Weights Model Loss	4.20233963727951

(b) Shakespeare test set – performance loss/ compare to a baseline – random weights

Due to the Shakespeare dataset containing characters that are not present in the training dataset, when model 1 was tested using the unseen Shakespeare dataset, an error occurred outlined as a “KeyError” due to the presence of characters out-of-vocabulary. This is due to the unique characters that are in this dataset but not in the training dataset which are outlined in table 1 above. In order to combat this issue two options were developed in which (i) the unique characters are removed from the test set and replaced with spaces so that no out-of-vocabulary characters are present, (ii) or retrain the model on a character level vocabulary created by the union of the child speech training dataset and the Shakespeare test set. This ensures all possible characters are present in the vocabulary of the trained model while keeping the Shakespeare dataset unchanged and not leaking any Shakespeare data into the training set. The only change that will occur is these new characters will be assigned weights when the model initialises although due to none of these characters being present in the training dataset, these character’s parameters stay near random because they are never updated; therefore keeping the child like speech present in the trained model while allowing the model to be trained on unseen and unaltered Shakespeare test set.

Similar to the child speech test set, model 1 and a random weighted model were tested and their cross entropy loss was logged. It can be seen from the table that both models performed badly and the randomly weighted model performed better. This highlights that due to model 1 being trained on a child speech dataset, making it very confident on child style character sequences. The unseen characters which were added to the training vocab to avoid errors are assigned very small weights which leads to them not being predicted, as well as this the vocabulary in the Shakespeare test contains words which are not in the scope of a child vocabulary.

The random model essentially behaves as a uniform predictor over the 65 character vocabulary which is highlighted by its loss of 4.19 ($\log(65) = 4.17$), causing the model to be equally unsure of every character. This demonstrates that using a trained model which is extremely out of domain can perform worse than a model which is assigned random weights.

The pipeline in which a model is trained and tested in very different datasets would be applicable in a situation where a user desires a model which can approximate how closely a test text corpus approximates the language domain the model has been trained on.

Table 6: Cross Entropy Loss for Model 1 tested on Shakespeare Speech Dataset

Trained Model Loss	8.321168112754822
Random Weights Model Loss	4.190468637943268

Code

```
import torch

import torch.nn as nn

from torch.nn import functional as F

import csv

import math

import json, pathlib


print("cuda available:", torch.cuda.is_available())

if torch.cuda.is_available():

    print("gpu:", torch.cuda.get_device_name(0))


USE_RESIDUAL_ATTN = True # residual around self-attention
USE_RESIDUAL_FFNN = True # residual around feed-forward
USE_PRENORM = False # True:  $x = x + SA(LN(x))$  ; False:  $x = LN(x + SA(x))$  (Post-LN)


# hyperparameters

batch_size = 64 # how many independent sequences will we process in parallel?

block_size = 256 # what is the maximum context length for predictions?

max_iters = 5000

eval_interval = 500

learning_rate = 3e-4

device = 'cuda' if torch.cuda.is_available() else 'cpu'

eval_iters = 200

n_embd = 128 #128 , 160 , 96
```

```

n_head = 4 #4,4,4
n_layer = 3 #3,2,6
dropout = 0.3
# -----

torch.manual_seed(1337)

# --- load all raw texts ---
with open('input_childSpeech_trainingSet.txt', 'r', encoding='utf-8') as f:
    child_train_text = f.read()

with open('input_childSpeech_testSet.txt', 'r', encoding='utf-8') as f:
    child_test_text = f.read()

with open('input_shakespeare.txt', 'r', encoding='utf-8') as f:
    shakespeare_text = f.read()

# --- build vocabulary from the UNION of all datasets ---
all_text = child_train_text + child_test_text + shakespeare_text
chars = sorted(list(set(all_text))) # CHANGED
vocab_size = len(chars)
print(f'vocab size (union): {vocab_size}')
print(f'Length of child TRAIN in characters: {len(child_train_text)}')

# create a mapping from characters to integers
stoi = { ch:i for i,ch in enumerate(chars) }
itos = { i:ch for i,ch in enumerate(chars) }
encode = lambda s: [stoi[c] for c in s] # encoder: take a string, output a list of integers
decode = lambda l: ''.join([itos[i] for i in l]) # decoder: take a list of integers, output a string

# Train and test splits

```

```
data = torch.tensor(encode(child_train_text), dtype=torch.long) # CHANGED
```

```
n = int(0.9*len(data)) # first 90% will be train, rest val
```

```
train_data = data[:n]
```

```
val_data = data[n:]
```

```
# data loading
```

```
def get_batch(split):
```

```
    # generate a small batch of data of inputs x and targets y
```

```
    data = train_data if split == 'train' else val_data
```

```
    ix = torch.randint(len(data) - block_size, (batch_size,))
```

```
    x = torch.stack([data[i:i+block_size] for i in ix])
```

```
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
```

```
    x, y = x.to(device), y.to(device)
```

```
    return x, y
```

```
@torch.no_grad()
```

```
def estimate_loss():
```

```
    out = {}
```

```
    model.eval()
```

```
    for split in ['train', 'val']:
```

```
        losses = torch.zeros(eval_iters)
```

```
        for k in range(eval_iters):
```

```
            X, Y = get_batch(split)
```

```
            logits, loss = model(X, Y)
```

```
            losses[k] = loss.item()
```

```
        out[split] = losses.mean()
```

```
    model.train()
```

```
    return out
```

```
class Head(nn.Module):
```

```
    """ one head of self-attention """
```

```

def __init__(self, head_size):
    super().__init__()
    self.key = nn.Linear(n_embd, head_size, bias=False)
    self.query = nn.Linear(n_embd, head_size, bias=False)
    self.value = nn.Linear(n_embd, head_size, bias=False)
    self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))

    self.dropout = nn.Dropout(dropout)

def forward(self, x):
    # input of size (batch, time-step, channels)
    # output of size (batch, time-step, head size)
    B,T,C = x.shape
    k = self.key(x) # (B,T,hs)
    q = self.query(x) # (B,T,hs)
    # compute attention scores ("affinities")
    wei = q @ k.transpose(-2,-1) * k.shape[-1]**-0.5 # (B, T, hs) @ (B, hs, T) -> (B, T, T)
    wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # (B, T, T)
    wei = F.softmax(wei, dim=-1) # (B, T, T)
    wei = self.dropout(wei)
    # perform the weighted aggregation of the values
    v = self.value(x) # (B,T,hs)
    out = wei @ v # (B, T, T) @ (B, T, hs) -> (B, T, hs)
    return out

class MultiHeadAttention(nn.Module):
    """ multiple heads of self-attention in parallel """

    def __init__(self, num_heads, head_size):
        super().__init__()

```

```

self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])

self.proj = nn.Linear(head_size * num_heads, n_embd)

self.dropout = nn.Dropout(dropout)

def forward(self, x):
    out = torch.cat([h(x) for h in self.heads], dim=-1)
    out = self.dropout(self.proj(out))
    return out

class FeedFoward(nn.Module):
    """ a simple linear layer followed by a non-linearity """

    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)

class Block(nn.Module):
    """ Transformer block: communication followed by computation """

    def __init__(self, n_embd, n_head):
        # n_embd: embedding dimension, n_head: the number of heads we'd like
        super().__init__()
        head_size = n_embd // n_head

```



```
self.sa = MultiHeadAttention(n_head, head_size)
```

```
self.ffwd = FeedFoward(n_embd)
```

```
self.ln1 = nn.LayerNorm(n_embd)
```

```
self.ln2 = nn.LayerNorm(n_embd)
```

```
def forward(self, x):
```

```
    if USE_PRENORM:
```

```
        # Pre-LN (your current default)
```

```
        attn_in = self.ln1(x)
```

```
        attn_out = self.sa(attn_in)
```

```
        x = x + attn_out if USE_RESIDUAL_ATTN else attn_out
```

```
        ffn_in = self.ln2(x)
```

```
        ffn_out = self.ffwd(ffn_in)
```

```
        x = x + ffn_out if USE_RESIDUAL_FFNN else ffn_out
```

```
    else:
```

```
        # Post-LN variant
```

```
        attn_out = self.sa(x)
```

```
        x = x + attn_out if USE_RESIDUAL_ATTN else attn_out
```

```
        x = self.ln1(x)
```

```
        ffn_out = self.ffwd(x)
```

```
        x = x + ffn_out if USE_RESIDUAL_FFNN else ffn_out
```

```
        x = self.ln2(x)
```

```
    return x
```

```
class GPTLanguageModel(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        # each token directly reads off the logits for the next token from a lookup table
```

```

self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
self.position_embedding_table = nn.Embedding(block_size, n_embd)
self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
self.ln_f = nn.LayerNorm(n_embd) # final layer norm
self.lm_head = nn.Linear(n_embd, vocab_size)

# better init, not covered in the original GPT video, but important, will cover in followup video
self.apply(self._init_weights)

def _init_weights(self, module):
    if isinstance(module, nn.Linear):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
        if module.bias is not None:
            torch.nn.init.zeros_(module.bias)
    elif isinstance(module, nn.Embedding):
        torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

def forward(self, idx, targets=None):
    B, T = idx.shape

    # idx and targets are both (B,T) tensor of integers
    tok_emb = self.token_embedding_table(idx) # (B,T,C)
    pos_emb = self.position_embedding_table(torch.arange(T, device=device)) # (T,C)
    x = tok_emb + pos_emb # (B,T,C)
    x = self.blocks(x) # (B,T,C)
    x = self.ln_f(x) # (B,T,C)
    logits = self.lm_head(x) # (B,T,vocab_size)

    if targets is None:
        loss = None
    else:

```

```

    B, T, C = logits.shape
    logits = logits.view(B*T, C)
    targets = targets.view(B*T)
    loss = F.cross_entropy(logits, targets)

    return logits, loss

def generate(self, idx, max_new_tokens):
    # idx is (B, T) array of indices in the current context
    for _ in range(max_new_tokens):
        # crop idx to the last block_size tokens
        idx_cond = idx[:, -block_size:]
        # get the predictions
        logits, loss = self(idx_cond)
        # focus only on the last time step
        logits = logits[:, -1, :] # becomes (B, C)
        # apply softmax to get probabilities
        probs = F.softmax(logits, dim=-1) # (B, C)
        # sample from the distribution
        idx_next = torch.multinomial(probs, num_samples=1) # (B, 1)
        # append sampled index to the running sequence
        idx = torch.cat((idx, idx_next), dim=1) # (B, T+1)
    return idx

model = GPTLanguageModel()
m = model.to(device)

# print the number of parameters in the model
print(sum(p.numel() for p in m.parameters())/1e6, 'M parameters')

# create a PyTorch optimizer
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

```

```

#added myself - to make graphs of train and val loss at each step

train_losses = []
val_losses = []
step_list = []
log_train_ppl, log_val_ppl = [], []

for iter in range(max_iters):

    # every once in a while evaluate the loss on train and val sets
    if iter % eval_interval == 0 or iter == max_iters - 1:
        losses = estimate_loss()
        #store losses for step iter
        train_loss = float(losses['train'])
        val_loss = float(losses['val'])
        train_ppl = math.exp(train_loss)
        val_ppl = math.exp(val_loss)

        print(f"step {iter}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")

        train_losses.append(train_loss)
        val_losses.append(val_loss)
        step_list.append(iter)
        log_train_ppl.append(train_ppl);
        log_val_ppl.append(val_ppl)

    # sample a batch of data
    xb, yb = get_batch('train')

    # evaluate the loss
    logits, loss = model(xb, yb)

```

```

optimizer.zero_grad(set_to_none=True)

loss.backward()

optimizer.step()


#generate from the model

context = torch.zeros((1, 1), dtype=torch.long, device=device)

print(decode(m.generate(context, max_new_tokens=500)[0].tolist()))

#open('more.txt', 'w').write(decode(m.generate(context, max_new_tokens=10000)[0].tolist()))


#write each models data for loss graphs to csv files

with open('gpt_train_losses_model1_no_prenorm.csv', mode='w', newline='') as f:

    writer = csv.writer(f)

    writer.writerow(['step', 'train_loss', 'val_loss', 'train_ppl', 'val_ppl'])

    for step, train_loss, val_loss, trn_ppl, vp in zip(step_list, train_losses, val_losses, log_train_ppl,
log_val_ppl):

        writer.writerow([step, train_loss, val_loss, trn_ppl, vp])


#save the model after training

#import json, pathlib


#after training loop finishes

#ckpt_path = "model1_best.pt"

#torch.save({

# "state_dict": model.state_dict(),

# "config": {

# "USE_RESIDUAL_ATTN": USE_RESIDUAL_ATTN,

# "USE_RESIDUAL_FFN": USE_RESIDUAL_FFN,

# "USE_PRENORM": USE_PRENORM,

# "vocab_size": vocab_size,

# "block_size": block_size,

# "n_embd": n_embd,

```

```

#   "n_head": n_head,
#   "n_layer": n_layer,
#   "dropout": dropout,
# },
#   "chars": chars, # to rebuild stoi/itos
#}, ckpt_path)
#
#print(f"saved checkpoint -> {ckpt_path}")

import torch
import torch.nn as nn
from torch.nn import functional as F
import math
import csv

print("cuda available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("gpu:", torch.cuda.get_device_name(0))

device = 'cuda' if torch.cuda.is_available() else 'cpu'

#globals from gpt.py
USE_RESIDUAL_ATTN = True
USE_RESIDUAL_FFNN = True
USE_PRENORM = True

block_size = 256
n_embd = 128
n_head = 4
n_layer = 3
dropout = 0.3

```

```
vocab_size = 0
```

```
torch.manual_seed(1337)
```

```
#takem from gpt.py
```

```
class Head(nn.Module):
```

```
    """ one head of self-attention """
```

```
    def __init__(self, head_size):
```

```
        super().__init__()
```

```
        self.key = nn.Linear(n_embd, head_size, bias=False)
```

```
        self.query = nn.Linear(n_embd, head_size, bias=False)
```

```
        self.value = nn.Linear(n_embd, head_size, bias=False)
```

```
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))
```

```
        self.dropout = nn.Dropout(dropout)
```

```
    def forward(self, x):
```

```
        # input of size (batch, time-step, channels)
```

```
        # output of size (batch, time-step, head size)
```

```
        B,T,C = x.shape
```

```
        k = self.key(x) # (B,T,hs)
```

```
        q = self.query(x) # (B,T,hs)
```

```
        # compute attention scores ("affinities")
```

```
        wei = q @ k.transpose(-2,-1) * k.shape[-1]**-0.5 # (B, T, hs) @ (B, hs, T) -> (B, T, T)
```

```
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # (B, T, T)
```

```
        wei = F.softmax(wei, dim=-1) # (B, T, T)
```

```
        wei = self.dropout(wei)
```

```
        # perform the weighted aggregation of the values
```

```
        v = self.value(x) # (B,T,hs)
```

```

out = wei @ v # (B, T, T) @ (B, T, hs) -> (B, T, hs)

return out

```

```

class MultiHeadAttention(nn.Module):
    """ multiple heads of self-attention in parallel """

    def __init__(self, num_heads, head_size):
        super().__init__()

        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])

        self.proj = nn.Linear(head_size * num_heads, n_embd)

        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)

        out = self.dropout(self.proj(out))

        return out


class FeedFoward(nn.Module):
    """ a simple linear layer followed by a non-linearity """

    def __init__(self, n_embd):
        super().__init__()

        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)

```



```

class Block(nn.Module):

    """ Transformer block: communication followed by computation """

    def __init__(self, n_embd, n_head):
        # n_embd: embedding dimension, n_head: the number of heads we'd like
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedFoward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        if USE_PRENORM:
            #Pre-LN
            attn_in = self.ln1(x)
            attn_out = self.sa(attn_in)
            x = x + attn_out if USE_RESIDUAL_ATTN else attn_out

            ffn_in = self.ln2(x)
            ffn_out = self.ffwd(ffn_in)
            x = x + ffn_out if USE_RESIDUAL_FFNN else ffn_out
        else:
            #post-LN
            attn_out = self.sa(x)
            x = x + attn_out if USE_RESIDUAL_ATTN else attn_out
            x = self.ln1(x)

            ffn_out = self.ffwd(x)
            x = x + ffn_out if USE_RESIDUAL_FFNN else ffn_out

```

```

    x = self.ln2(x)
    return x

```

```

class GPTLanguageModel(nn.Module):

```

```

    def __init__(self):
        super().__init__()
        # each token directly reads off the logits for the next token from a lookup table
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd) # final layer norm
        self.lm_head = nn.Linear(n_embd, vocab_size)

        # better init, not covered in the original GPT video, but important, will cover in followup video
        self.apply(self._init_weights)

```

```

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                torch.nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

```

```

    def forward(self, idx, targets=None):
        B, T = idx.shape

        # idx and targets are both (B,T) tensor of integers
        tok_emb = self.token_embedding_table(idx) # (B,T,C)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device)) # (T,C)

```

```

x = tok_emb + pos_emb # (B,T,C)
x = self.blocks(x) # (B,T,C)
x = self.ln_f(x) # (B,T,C)
logits = self.lm_head(x) # (B,T,vocab_size)

if targets is None:
    loss = None
else:
    B, T, C = logits.shape
    logits = logits.view(B*T, C)
    targets = targets.view(B*T)
    loss = F.cross_entropy(logits, targets)

return logits, loss

def generate(self, idx, max_new_tokens):
    # idx is (B, T) array of indices in the current context
    for _ in range(max_new_tokens):
        # crop idx to the last block_size tokens
        idx_cond = idx[:, -block_size:]
        # get the predictions
        logits, loss = self(idx_cond)
        # focus only on the last time step
        logits = logits[:, -1, :] # becomes (B, C)
        # apply softmax to get probabilities
        probs = F.softmax(logits, dim=-1) # (B, C)
        # sample from the distribution
        idx_next = torch.multinomial(probs, num_samples=1) # (B, 1)
        # append sampled index to the running sequence
        idx = torch.cat((idx, idx_next), dim=1) # (B, T+1)
    return idx

```

```
#load checkpoint from model 1 and get hyperparams
```

```
ckpt_path = "model1_best.pt"
```

```
ckpt = torch.load(ckpt_path, map_location=device)
```

```
print(f"loaded checkpoint model {ckpt_path}")
```

```
cfg = ckpt["config"]
```

```
#overwrite the training configs to match the loaded model
```

```
USE_RESIDUAL_ATTN = cfg["USE_RESIDUAL_ATTN"]
```

```
USE_RESIDUAL_FFNN = cfg["USE_RESIDUAL_FFNN"]
```

```
USE_PRENORM = cfg["USE_PRENORM"]
```

```
vocab_size = cfg["vocab_size"]
```

```
block_size = cfg["block_size"]
```

```
n_embd = cfg["n_embd"]
```

```
n_head = cfg["n_head"]
```

```
n_layer = cfg["n_layer"]
```

```
dropout = cfg["dropout"]
```

```
chars = ckpt["chars"]
```

```
#encoder/decoder from saved chars
```

```
stoi = { ch:i for i, ch in enumerate(chars) }
```

```
itos = { i:ch for i, ch in enumerate(chars) }
```

```
def encode(s):
```

```
    return [stoi[c] for c in s]
```

```
def decode(indices):
```

```
    return ''.join(itos[i] for i in indices)
```

```

#instantiate model with correct configs
model = GPTLanguageModel().to(device)
model.load_state_dict(ckpt["state_dict"])
model.eval()

print(sum(p.numel() for p in model.parameters())/1e6, "M parameters")

#load test texts
with open('input_childSpeech_testSet.txt', 'r', encoding='utf-8') as f:
    child_test_text = f.read()

with open('input_shakespeare.txt', 'r', encoding='utf-8') as f:
    shakespeare_text = f.read()

#evaluation helper
@torch.no_grad()
def eval_loss_on_text(raw_text, label="", eval_iters=200, batch_size=64):
    ##estimate average loss and ppl on text
    data = torch.tensor(encode(raw_text), dtype=torch.long)

    losses = []
    for _ in range(eval_iters):
        ix = torch.randint(0, len(data) - block_size - 1, (batch_size,))
        x = torch.stack([data[i:i+block_size] for i in ix])
        y = torch.stack([data[i+1:i+block_size+1] for i in ix])
        x, y = x.to(device), y.to(device)
        _, loss = model(x, y)
        losses.append(loss.item())

    avg_loss = sum(losses) / len(losses)
    ppl = math.exp(avg_loss)

```

```

    print(f"[{label}] loss: {avg_loss:.4f}, ppl: {ppl:.2f}")
    return avg_loss, ppl

results = []

child_loss, child_ppl = eval_loss_on_text(child_test_text, label="child_test")
results.append(("child_test", child_loss, child_ppl))

shak_loss, shak_ppl = eval_loss_on_text(shakespeare_text, label="shakespeare")
results.append(("shakespeare", shak_loss, shak_ppl))

##save results to csv
#with open("gpt_eval_results_model1.csv", "w", newline="") as f:
#    writer = csv.writer(f)
#    writer.writerow(["dataset", "loss", "ppl"])
#    for name, loss, ppl in results:
#        writer.writerow([name, loss, ppl])
#
#print("Saved evaluation results -> gpt_eval_results_model1.csv")

#get performance on both datasets with random weight model to compare

#load settings from checkpoint from model 1 to get hyperparams to keep them constant
#between 2 models
ckpt_path = "model1_best.pt"
ckpt = torch.load(ckpt_path, map_location=device)
print(f"loaded checkpoint model {ckpt_path}")

cfg = ckpt["config"]

#overwrite the training configs to match the loaded model

```

```

USE_RESIDUAL_ATTN = cfg["USE_RESIDUAL_ATTN"]
USE_RESIDUAL_FFN = cfg["USE_RESIDUAL_FFN"]
USE_PRENORM = cfg["USE_PRENORM"]
vocab_size = cfg["vocab_size"]
block_size = cfg["block_size"]
n_embd = cfg["n_embd"]
n_head = cfg["n_head"]
n_layer = cfg["n_layer"]
dropout = cfg["dropout"]

#reconstrcut vocab from saved chars
stoi = { ch:i for i, ch in enumerate(chars) }
itos = { i:ch for i, ch in enumerate(chars) }

def encode(s):
    return [stoi[c] for c in s]

def decode(indices):
    return ''.join(itos[i] for i in indices)

#load tests set
with open('input_childSpeech_testSet.txt', 'r', encoding='utf-8') as f:
    child_test_text = f.read()

with open('input_shakespeare.txt', 'r', encoding='utf-8') as f:
    shakespeare_text = f.read()

#eval loss and ppl
#take model and raw text as input

@torch.no_grad()

```

```

def eval_loss_on_text(model, raw_text, label="", eval_iters=200, batch_size=64):

    model.eval()

    #encode raw text to tensor
    data = torch.tensor(encode(raw_text), dtype=torch.long)

    losses = []

    #for every batch get random sample and record cross entropy loss
    for _ in range(eval_iters):

        ix = torch.randint(0, len(data) - block_size - 1, (batch_size,))

        x = torch.stack([data[i:i+block_size] for i in ix])

        y = torch.stack([data[i+1:i+block_size+1] for i in ix])

        x, y = x.to(device), y.to(device)

        _, loss = model(x, y)

        losses.append(loss.item())

    avg_loss = sum(losses) / len(losses)

    ppl = math.exp(avg_loss)

    print(f"[{label}] loss: {avg_loss:.4f}, ppl: {ppl:.2f}")

    return avg_loss, ppl

# 1) Random-weight model

torch.manual_seed(1337)

model_rand = GPTLanguageModel().to(device)

print(sum(p.numel() for p in model_rand.parameters())/1e6, "M parameters (random model)")

rand_child_loss, rand_child_ppl = eval_loss_on_text(model_rand, child_test_text,
label="RANDOM_child_test"

)

rand_shake_loss, rand_shake_ppl = eval_loss_on_text(model_rand, shakespeare_text,
label="RANDOM_shakespeare"

)

```



```

# 2) Trained model (Model 1)

model_trained = GPTLanguageModel().to(device)
model_trained.load_state_dict(ckpt["state_dict"])
print(sum(p.numel() for p in model_trained.parameters())/1e6, "M parameters (trained model)")

child_loss, child_ppl = eval_loss_on_text(
    model_trained, child_test_text, label="TRAINED_child_test"
)
shake_loss, shake_ppl = eval_loss_on_text(
    model_trained, shakespeare_text, label="TRAINED_shakespeare"
)

#save to csv
results = [
    ("random","child_test",rand_child_loss,rand_child_ppl),
    ("random","shakespeare",rand_shake_loss,rand_shake_ppl),
    ("trained","child_test",child_loss,child_ppl),
    ("trained","shakespeare",shake_loss,shake_ppl),
]

with open("gpt_eval_results_model1_random_vs_trained.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["model_type", "dataset", "loss", "ppl"])
    for row in results:
        writer.writerow(row)

print("Saved evaluation results gpt_eval_results_model1_random_vs_trained.csv")

import csv
import matplotlib.pyplot as plt

```

```

#
def load_csv_loss_data(filename):
    steps = []
    train_losses = []
    val_losses = []
    train_ppls = []
    val_ppls = []
    with open(filename, mode='r') as f:
        reader = csv.reader(f)
        next(reader) # skip header
        for row in reader:
            steps.append(int(row[0]))
            train_losses.append(float(row[1]))
            val_losses.append(float(row[2]))
            #train_ppls.append(float(row[3]))
            #val_ppls.append(float(row[4]))
    return steps, train_losses, val_losses

#plot data
def plot_loss_per_step(steps, train_losses, val_losses, model_label):
    plt.plot(steps, train_losses, label='Train Loss')
    plt.plot(steps, val_losses, label='Validation Loss')
    plt.xlabel('Steps')
    plt.ylabel('Loss')
    plt.title(f'Training and Validation Loss per Step - {model_label}')
    plt.legend()
    plt.grid()
    plt.show(
)

def plot_perplexity_per_step(steps, train_ppls, val_ppls, model_label):

```

```

plt.plot(steps, train_ppls, label='Train Perplexity')
plt.plot(steps, val_ppls, label='Validation Perplexity')
plt.xlabel('Steps')
plt.ylabel('Perplexity')
plt.title(f'Training and Validation Perplexity per Step - {model_label}')
plt.legend()
plt.grid()
plt.show()

#main to run functions
if __name__ == "__main__":
    #
    # Load data from CSV files
    steps1, train_losses1, val_losses1 =
load_csv_loss_data(r'C:\Users\irish\Computer_Electronic_Engineering_Year5\Machine_Learnin
g\Week_9\model_data\model_3\gpt_train_losses_model3.csv')

    # Plot the loss data for both models
    plot_loss_per_step(steps1, train_losses1, val_losses1, model_label='Model 3')

    #plot_perplexity_per_step(steps1, train_ppls1, val_ppls1, model_label='Model 1 with PreLN,
no ResAttn, with ResFfn')

```

